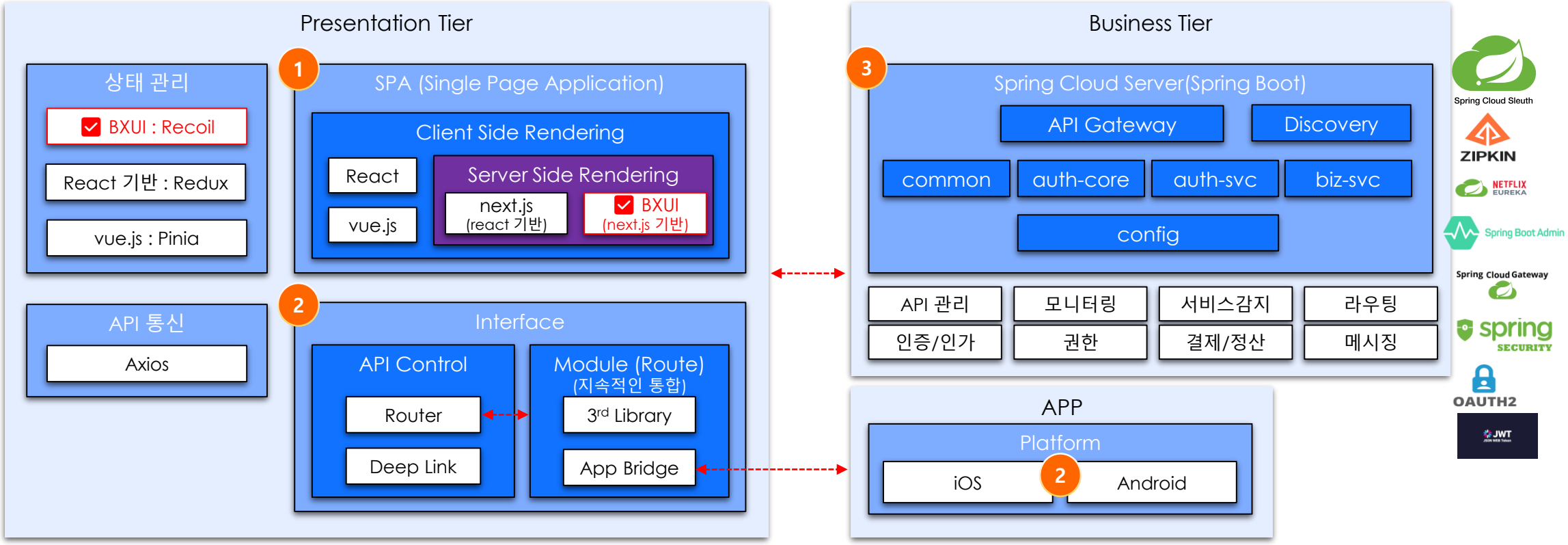
채널 프레임워크 구축

Ver 0.6

bankware global

채널 프레임워크 구성



1
1. BXUI : 퍼블리셔 팀 지원 (1순위)
2. next.js : BXUI의 기반이 next.js (2순위)
3. react : next.js의 기반이 react
4. vue.js : 같은 SPA 이지만, 다른 제품 (샘플 처리)

2
1. 목적 : 필요한 기능들을 모듈화 하여 **API Control**에 의해 동작 되도록 처리 **(앱 연동 포함)**
2. 경쟁력 : **3rd Library** 들을 미리 통합하여, 타사 SI 업체에 비해 빠른 일정에 처리
3. 필요사항 : **3rd Library** 업체를 통한 Library확보 및 기술 지원 필요 (구 페이지 → SPA 변경)

3
Spring Cloud Server 기술 기반 구조 : MSA 아키텍처 구축 시 필요한 공통 문제들을 해결해주는 Spring 기술
1. API Gateway/라우팅 : 클라이언트 요청을 적절한 서비스로 전달
2. 서비스 등록/발견(Eureka) : 서비스 인스턴스가 위치 관리
3. 부하 분산/장애처리/모니터링 : 고도화 시 확장
4. 향후 경량화 버전 백엔드 추가 : node.js 고려

Presentation Tier (Front-End)

인증

업체	카카오싸인	바로써트 (구.카카오써트)
통합 인증	카카오 / 네이버 / 패스 / 토스	카카오 / 네이버 / 패스

정리할 인증
1. 사설인증 (간편인증)
    1. 카카오 / 네이버
2. 키패트 (웹, 앱)
3. 공동인증서 (웹, 앱)
4. 생체인증서 (앱 기반 처리)

요금안내

카카오써트는 고객사의 다양한 운영방식을 고려하여 후불로 서비스를 제공하고 있으며, 사용하는 인증 서비스가 많아질수록 결합할인 혜택이 늘어나 저렴하게 이용 가능합니다.

구.카카오서트 요금표

*부가세별도

구분	서비스	서비스 코드	기본요금	결제시점
인증	본인인증	K1110/K1120	30원/건	모든 기업 조건없이 후불지원
	전자서명(단건)	K3510/K3520	35원/건	
	전자서명(복수)	K3511/K3521	80원/건	
	출금동의	K3110/K3120	35원/건	
전자문서	전자우편	D10/D11	150원/건	
	간편문서	D20/D21	50원/건	
	전자서명문서	D31	150원/건	

Business Tier (Back-End)

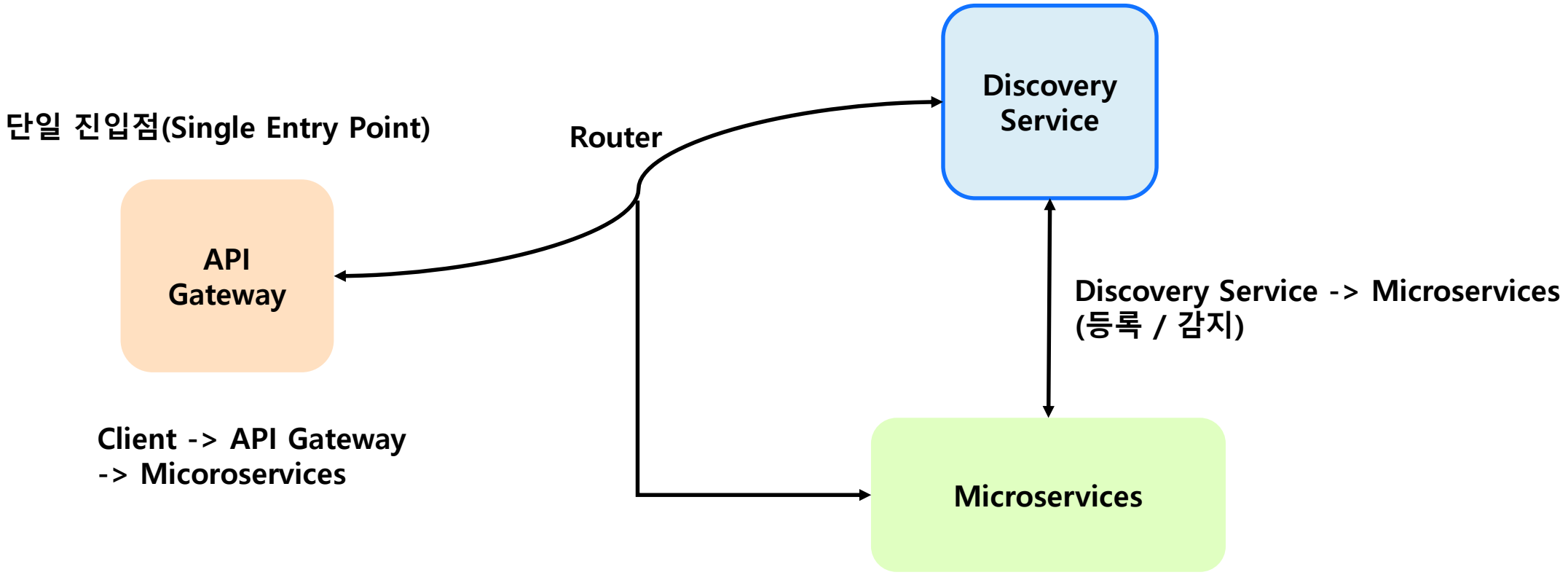
프로젝트 구조 (monorepo)

```
bx-cf-be/
project-root/
│
├── build.gradle # root gradle (전체 dependency 관리 가능)
├── settings.gradle # 모듈 include
├── gradle/ # gradle wrapper
├── src/
│   └── main/
│   └── resources/
│   └── config/ # 각프로젝트 설정, 공통 리소스
│
├── common/ # 공통 모듈
│   ├── build.gradle
│   └── src/
│   └── main/
│   ├── java/com/bwg/channel/backend/common/
│   │   ├── dto/
│   │   ├── exception/
│   │   └── util/
│   └── resources/
│   └── application.yml
│
├── auth-core/ # 인증/인가 공통
│   ├── build.gradle
│   └── src/
│   └── main/
│   ├── java/com/bwg/channel/backend/authcore/
│   │   ├── filter/
│   │   ├── provider/
│   │   ├── service/
│   │   └── util/
```

```
├── auth-svc/ # 인증/인가 서비스
│   ├── build.gradle
│   └── src/
│   └── main/
│   ├── java/com/bwg/channel/backend/authsvc/
│   │   ├── AuthServiceApplication.java
│   │   ├── cmm/
│   │   ├── controller/
│   │   ├── service/
│   │   ├── repository/
│   │   └── security/
│
├── product-svc/ # 상품 관리 서비스(가칭)
│   ├── build.gradle
│   └── src/
│   └── main/
│   ├── java/com/bwg/channel/backend/product/
│   │   ├── ProductServiceApplication.java
│   │   └── cmm/
│   │   ├── controller/
│   │   ├── service/
│   │   ├── repository/
│
├── biz-svc/ # 업무 서비스(가칭)
│   ├── build.gradle
│   └── src/
│   └── main/
│   ├── java/com/bwg/channel/backend/biz/
│   │   ├── BizServiceApplication.java
│   │   └── cmm/
│   │   ├── controller/
│   │   ├── service/
│   │   ├── repository/
```

```
├── api-gateway/ # API Gateway
│   ├── build.gradle
│   └── src/
│   └── main/
│   ├── java/com/bwg/channel/backend/gateway/
│   │   ├── ApiGatewayApplication.java
│   │   └── cmm/
│
│
└── discovery-svc/ # Service Discovery (Eureka)
├── build.gradle
└── src/
└── main/
├── java/com/bwg/channel/backend/discovery/
│   └── DiscoveryServiceApplication.java
```

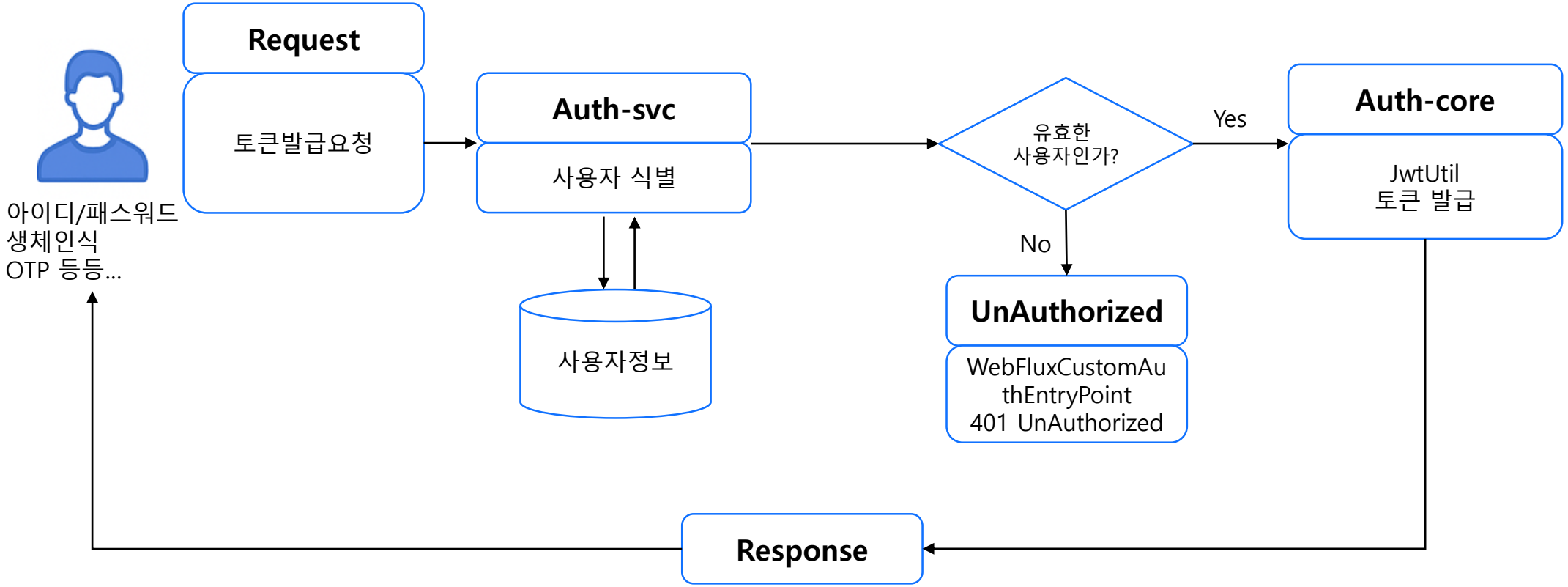
Spring Cloud Server



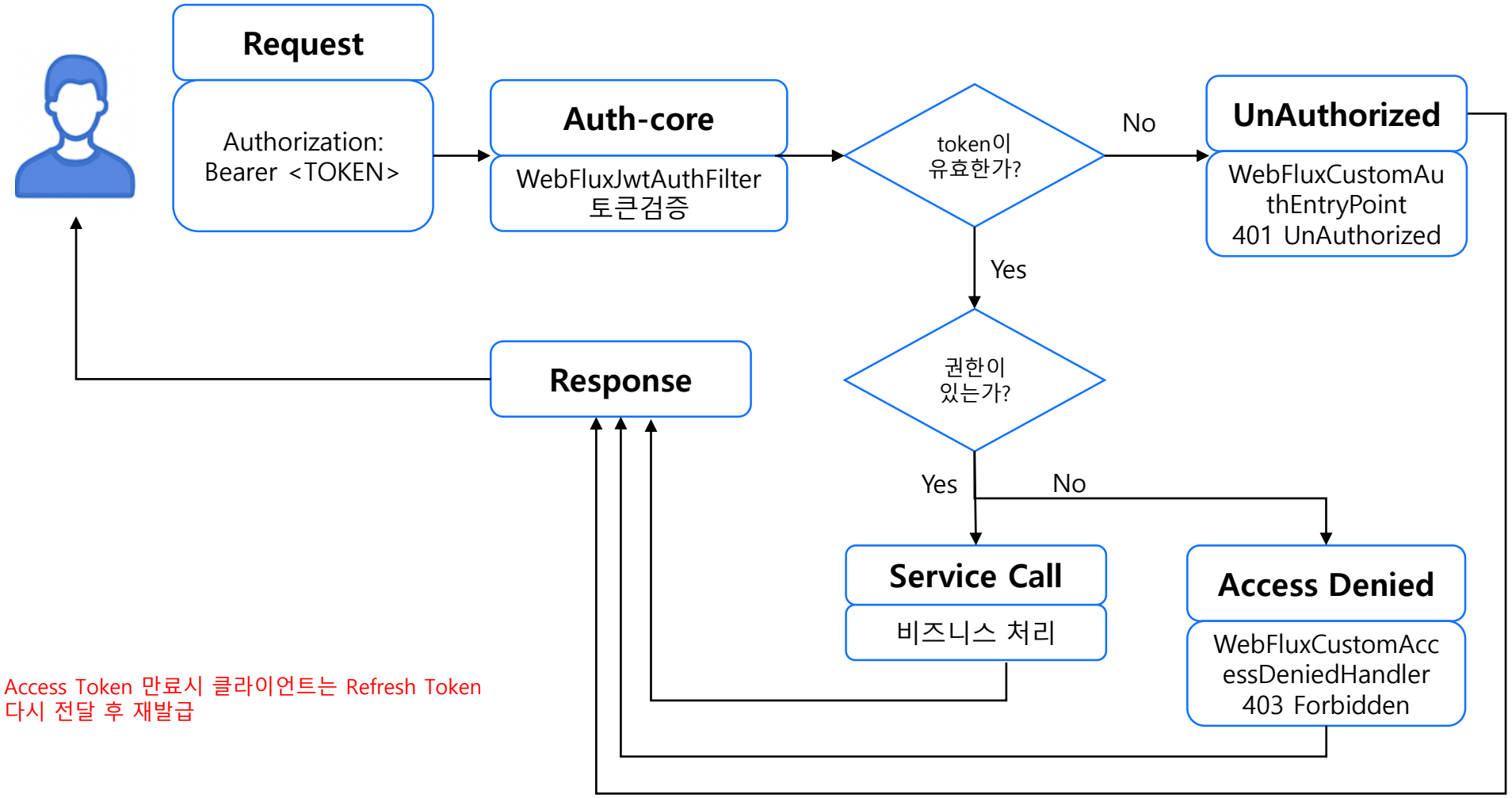
- API Gateway – 내외부를 잇는 관문
- Discovery Service – 주소록 & 교환
- Microserivces – 독립 비즈니스 로직 담당 (auth-svc · product-svc)

* 서로 긴밀히 연결되어 서비스 전체의 유연성과 확장성을 책임진다.

API Gateway - 토큰발급요청(auth-svc)



API Gateway – 토큰검증 인증/인가(auth-svc)



Access Token 만료시 클라이언트는 Refresh Token
다시 전달 후 재발급

bankware global

설정 관리 – Profile-based YML

디렉토리 구조

```
src/main/resources/config/
  ├─ {module}/
  │    application.yml            ← 모듈 공통 (항상 포함)
  ├─ {module}/local/
  │    application-local.yml      ← local 전용
  └─ {module}/dev/
       application-dev.yml        ← dev 전용

  └─ auth-core/{profile}/
       application-authcore-{profile}.yml
```

빌드 동작 방식

processResources가 -Pprofile 값에 따라
해당 환경의 YML 파일만 JAR에 패키징

→ 다른 환경 DB 정보가 JAR에 포함되지 않음 (보안)

항목	local	dev
Eureka	localhost	192.168.11.197
Token TTL	1 분	30 분
Log	(콘솔)	~/apps/.../logs
DB	192.168.11.197	192.168.11.197

$ 빌드 명령

```
./gradlew :services:auth-svc:bootJar -Pprofile=dev
./gradlew :services:product-svc:bootJar -Pprofile=dev
```

결과물 → `build/dist/bx-cf-be/*.jar`

⚠ 주의

모듈 자체 resources에 동일 파일이 있으면 EXCLUDE 전략으로 공통 설정이 무시됨

API 목록 & Swagger 집계

Client → API Gateway :18081 → auth-svc :18082 / product-svc :18083

auth-svc 엔드포인트

```
POST  /channel/backend/api/v1/auth/login
  body :  usrId, usrPwd (SHA-256)
  return:  accessToken, refreshToken

POST  /channel/backend/api/v1/auth/refresh-token
  body :  refreshToken
  return:  새 accessToken
```

★ 인증 불필요 (퍼블릭 엔드포인트)

product-svc 엔드포인트

```
GET  /channel/backend/api/v1/product/list
  ?useYn=Y&productNm=펀드
  return:  상품 목록

GET  /channel/backend/api/v1/product/{productId}
  return:  상품 단건
```

JWT 인증 필요 (Authorization: Bearer ...)

Swagger UI

:18081/swagger-ui.html

드롭다운으로 서비스 전환
→ auth-svc
→ product-svc

Authorize → Bearer 토큰 입력
→ 이후 요청에 자동 첨부
(persist-authorization 활성)

공통 응답 규격

```
{ "success": true, "code": "0", "msg": "success", "payload": { ... } }
```

bankware global

CI/CD – GitHub Actions + Self-hosted Runner

1
Push
develop 브랜치

2
Detect
paths-filter
경로 감지

3
Build JAR
./gradlew bootJar
-Pprofile=dev

4
Deploy
stop → start
스크립트 실행

🎯 선택적 재배포 규칙 (dorny/paths-filter)

변경 경로	→ 재배포 대상
`services/auth-svc/**`	→ `auth-svc` 만 재기동
`services/product-svc/**`	→ `product-svc` 만 재기동
`libs/**`	→ 전체 재기동
`infra/**`	→ 전체 재기동
`scripts/**`	→ 전체 재기동
`.github/workflows/**`	→ 전체 재기동

빌드 실패 시 → `stop` 실행 안 됨 → 기존 서비스 유지

macOS Runner 핵심 설정

env -u RUNNER_TRACKING_ID nohup java -jar ...

job 종료 후에도 프로세스가 살아남는다
(RUNNER_TRACKING_ID 제거로 kill 방지)

📜 배포 스크립트

```
scripts/start-all.sh   ← 기동 (discovery → svc → gw)
scripts/stop-all.sh    ← 종료 (역순: gw → svc → discovery)

JAR 위치:  ~/apps/bx-cf-be/{svc}/{svc}.jar
PID 위치:  ~/apps/bx-cf-be/{svc}/{svc}.pid
```

bankware global

환경별 접속 정보 – Local / Dev

Local 환경 (IntelliJ에서 직접 실행)

discovery-svc	:18761	Eureka Server
api-gateway	:18081	Spring Cloud Gateway
auth-svc	:18082	인증 / 토큰
product-svc	:18083	상품 관리

접속 주소

```
Eureka UI   →  http://localhost:18761
Swagger UI  →  http://localhost:18081/swagger-ui.html
API Base    →  http://localhost:18081/channel/backend/api/v1/
DB          →  jdbc:postgresql://192.168.11.197:5432/bxcfdb
```

실행 순서

discovery-svc → auth-svc + product-svc → api-gateway

Dev 환경 (macOS 서버: 192.168.11.197)

discovery-svc	:18761	Eureka Server
api-gateway	:18081	Spring Cloud Gateway
auth-svc	:18082	인증 / 토큰
product-svc	:18083	상품 관리

접속 주소 (nginx 80포트 경유)

```
Eureka UI   →  http://192.168.11.197:18761
Swagger UI  →  http://192.168.11.197:18081/swagger-ui.html
API Base    →  http://192.168.11.197/channel/backend/api/v1/
DB          →  jdbc:postgresql://192.168.11.197:5432/bxcfdb
```

서버 경로

```
JAR  →  ~/apps/bx-cf-be/{svc}/{svc}.jar
Log  →  ~/apps/bx-cf-be/logs/{svc}.log
```